

Bash Cheatsheet

By Dejan Panovski • Updated on Aug 17, 2026 •

Quick reference for Bash shell scripting

Bash (Bourne Again Shell) is the default shell on most Linux distributions. This cheatsheet covers essential Bash scripting concepts including variables, conditionals, loops, functions, and more. Perfect for writing automation scripts and working efficiently on the command line.

Script Basics

Start every Bash script with these fundamentals.

<code>#!/bin/bash</code>	Shebang (script interpreter)
<code>#!/usr/bin/env bash</code>	Portable shebang
<code># comment</code>	Single line comment
<code>chmod +x script.sh</code>	Make executable
<code>./script.sh</code>	Run script
<code>source script.sh</code>	Run in current shell
<code>. script.sh</code>	Same as source

Variables

Declare and use variables.

<code>var="value"</code>	Assign variable (no spaces!)
<code>\$var</code>	Use variable
<code>\${var}</code>	Use variable (explicit)
<code>readonly var</code>	Make read-only
<code>unset var</code>	Delete variable
<code>export var</code>	Export to environment
<code>env</code>	List environment vars

Special Variables

Built-in variables with special meanings.

<code>\$0</code>	Script name
<code>\$1 - \$9</code>	Positional parameters
<code>\${10}</code>	10th+ parameter
<code>\$#</code>	Number of arguments
<code>\$@</code>	All arguments (separate)
<code>\$*</code>	All arguments (single)
<code>\$?</code>	Last exit status
<code>\$\$</code>	Current PID
<code>\$_</code>	Last background PID

Quoting

Control variable expansion and special characters.

<code>"\$var"</code>	Double quotes (expands vars)
<code>'\$var'</code>	Single quotes (literal)
<code>\\$</code>	Escape special char
<code>`cmd`</code>	Command substitution (old)
<code>\$(cmd)</code>	Command substitution
<code>\$((expr))</code>	Arithmetic expansion

String Operations

Manipulate strings with parameter expansion.

<code>\${#var}</code>	String length
<code>\${var:0:5}</code>	Substring (pos 0, len 5)
<code>\${var#pattern}</code>	Remove prefix (shortest)
<code>\${var##pattern}</code>	Remove prefix (longest)
<code>\${var%pattern}</code>	Remove suffix (shortest)
<code>\${var%%pattern}</code>	Remove suffix (longest)
<code>\${var/old/new}</code>	Replace first
<code>\${var//old/new}</code>	Replace all

String Concatenation

Combine strings together.

<code><u>\$a\$b</u></code>	Concatenate variables
<code>"\${a}\${b}"</code>	Safe concatenation
<code>"\$a and \$b"</code>	With literal text
<code>var+="more"</code>	Append to variable

Default Values

Provide fallback values for variables.

<code>\${var:-default}</code>	Use default if unset
<code>\${var:=default}</code>	Set default if unset
<code>\${var:+value}</code>	Use value if set
<code>\${var:?error}</code>	Error if unset

Arrays

Work with indexed arrays.

<code><u>arr=(a b c)</u></code>	Declare array
<code>arr[0]="value"</code>	Set element
<code>\${arr[0]}</code>	Get element
<code>\${arr[@]}</code>	All elements
<code>\${#arr[@]}</code>	Array length
<code>\${!arr[@]}</code>	All indices
<code>arr+=(d e)</code>	Append elements
<code>unset arr[1]</code>	Delete element

Arithmetic

Perform math operations.

<code>\$((a + b))</code>	Addition
<code>\$((a - b))</code>	Subtraction
<code>\$((a * b))</code>	Multiplication
<code>\$((a / b))</code>	Division
<code>\$((a % b))</code>	Modulo
<code>\$((a ** b))</code>	Exponent
<code><u>((i++))</u></code>	Increment
<code>((i--))</code>	Decrement
<code>((i += 5))</code>	Add and assign

Conditionals

Test conditions and make decisions.

<code><u>if [[cond]]; then</u></code>	If statement
<code>elif [[cond]]; then</code>	Else if
<code>else</code>	Else clause
<code>fi</code>	End if
<code>[[cond]] && cmd</code>	Short-circuit and
<code>[[cond]] cmd</code>	Short-circuit or

Test Command

The `[[ ]]` test syntax (preferred over `[ ]`).

<code>[[-z \$str]]</code>	String is empty
<code>[[-n \$str]]</code>	String not empty
<code>[[\$a == \$b]]</code>	Strings equal
<code>[[\$a != \$b]]</code>	Strings not equal
<code>[[\$a =~ regex]]</code>	Regex match
<code>[[\$a == *pattern*]]</code>	Glob pattern match

Numeric Comparisons

Compare numbers in conditions.

<code>[[\$a -eq \$b]]</code>	Equal
<code>[[\$a -ne \$b]]</code>	Not equal
<code>[[\$a -lt \$b]]</code>	Less than
<code>[[\$a -le \$b]]</code>	Less or equal
<code>[[\$a -gt \$b]]</code>	Greater than
<code>[[\$a -ge \$b]]</code>	Greater or equal
<code>((a == b))</code>	Equal (arithmetic)
<code>((a < b))</code>	Less than (arithmetic)

File Tests

Test file properties.

<code>[[-e \$f]]</code>	File exists
<code>[[-f \$f]]</code>	Is regular file
<code>[[-d \$f]]</code>	Is directory
<code>[[-L \$f]]</code>	Is symlink
<code>[[-r \$f]]</code>	Is readable
<code>[[-w \$f]]</code>	Is writable
<code>[[-x \$f]]</code>	Is executable
<code>[[-s \$f]]</code>	Size > 0
<code>[[\$a -nt \$b]]</code>	a newer than b

Logical Operators

Combine multiple conditions.

<code>[[cond1 && cond2]]</code>	AND
<code>[[cond1 cond2]]</code>	OR
<code>[[! cond]]</code>	NOT
<code>[[(cond1)]]</code>	Grouping

Case Statement

Pattern matching with multiple branches.

<code>case \$var in</code>	Start case
<code>pattern)</code>	Match pattern
<code>a b)</code>	Match a or b
<code>*)</code>	Default case
<code>;;</code>	End pattern block
<code>esac</code>	End case

For Loop

Iterate over lists and ranges.

<code>for i in 1 2 3; do</code>	Loop over list
<code>for i in "\${arr[@]}"; do</code>	Loop over array
<code>for i in *.txt; do</code>	Loop over files
<code>for i in {1..10}; do</code>	Loop over range
<code>for ((i=0; i<10; i++)); do</code>	C-style for
<code>done</code>	End loop

While & Until Loops

Loop while/until condition is true.

<code>while [[cond]]; do</code>	While loop
<code>until [[cond]]; do</code>	Until loop
<code>while read -r line; do</code>	Read file lines
<code>while :; do</code>	Infinite loop
<code>done</code>	End loop
<code>done < file</code>	Read from file

Loop Control

Control loop execution flow.

<code>break</code>	Exit loop
<code>break 2</code>	Exit 2 levels
<code>continue</code>	Skip to next iteration
<code>continue 2</code>	Skip 2 levels

Functions

Define reusable code blocks.

<code>func() { }</code>	Define function
<code>function func { }</code>	Alternative syntax
<code>func arg1 arg2</code>	Call function
<code>\$1, \$2</code>	Function arguments
<code>local var="val"</code>	Local variable
<code>return 0</code>	Return exit code
<code>result=\$(func)</code>	Capture output

Input/Output

Read input and display output.

<code>echo "text"</code>	Print with newline
<code>echo -n "text"</code>	Print without newline
<code>echo -e "a\tb"</code>	Enable escapes
<code>printf "%s\n" "\$var"</code>	Formatted output
<code>read var</code>	Read into variable
<code>read -p "prompt: " var</code>	Read with prompt
<code>read -s var</code>	Read silently
<code>read -r var</code>	Raw input (no escapes)

Redirection

Redirect input and output streams.

<code>cmd > file</code>	Redirect stdout
<code>cmd >> file</code>	Append stdout
<code>cmd 2> file</code>	Redirect stderr
<code>cmd 2>&1</code>	Stderr to stdout
<code>cmd &> file</code>	Both to file
<code>cmd < file</code>	Redirect stdin
<code>cmd1 cmd2</code>	Pipe stdout

Here Documents

Embed multi-line text in scripts.

<code>cat <<EOF</code>	Here document
<code>cat <<'EOF'</code>	No expansion
<code>cat <<-EOF</code>	Strip leading tabs
<code>cmd <<< "string"</code>	Here string

Exit Codes

Handle command success and failure.

<code>exit 0</code>	Exit success
<code>exit 1</code>	Exit failure
<code>\$?</code>	Last exit code
<code>cmd && echo ok</code>	Run if success
<code>cmd echo fail</code>	Run if failure
<code>set -e</code>	Exit on error

Select Menu

Create interactive menus.

<code>select opt in a b c; do</code>	Create menu
<code>\$opt</code>	Selected option
<code>\$REPLY</code>	User's input
<code>break</code>	Exit menu
<code>done</code>	End select

Process Handling

Manage background processes.

<code>cmd &</code>	Run in background
<code>wait</code>	Wait for all jobs
<code>wait \$pid</code>	Wait for PID
<code>\$!</code>	Last background PID
<code>jobs</code>	List jobs
<code>fg %1</code>	Bring to foreground
<code>sleep 5</code>	Pause 5 seconds
<code>sleep 2m 30s</code>	Pause 2 min 30 sec
<code>sleep infinity</code>	Pause until interrupted

Debugging

Debug and trace script execution.

<code>bash -x script.sh</code>	Trace execution
<code>set -x</code>	Enable tracing
<code>set +x</code>	Disable tracing
<code>set -v</code>	Print input lines
<code>set -e</code>	Exit on error
<code>set -u</code>	Error on unset vars
<code>set -o pipefail</code>	Pipe error status

Useful Patterns

Common scripting patterns.

<code>\${var:-\$(cmd)}</code>	Default from command
<code>[[-f \$f]] && source \$f</code>	Source if exists
<code>while IFS= read -r line</code>	Read lines safely
<code>"\${arr[@]}"</code>	Safe array expansion
<code>set -- "\$@" newarg</code>	Append to args

Configuration Files

Bash startup files.

~/.bashrc	Interactive non-login
~/.bash_profile	Login shell
~/.profile	Generic login
~/.bash_aliases	Alias definitions
/etc/bash.bashrc	System-wide
/etc/profile	System login